

■ Chapter 1: Introduction to Operating Systems

1.1 Definition & Overview

An **Operating System (OS)** is system software that manages computer hardware, software resources, and provides common services for computer programs. It acts as an intermediary interface between computer hardware and the user applications.

1.2 Core Functions of an OS

- **Process Management:** Creation, scheduling, and termination of active processes.
- **Memory Management:** Allocation and deallocation of main memory (RAM) space.
- **File System Management:** Organization, storage, retrieval, and access control of files.
- **Device Management:** Coordinating I/O devices via hardware drivers.

1.3 Monolithic Kernel vs Microkernel

In a **Monolithic Kernel**, all system components such as the file system, memory manager, device drivers, and IPC run in kernel mode within a single address space. In contrast, a **Microkernel** keeps only core services (like IPC and basic scheduling) in kernel mode, while running drivers and file systems in user mode.

■ Chapter 2: Process Management & PCB

2.1 What is a Process?

A **Process** is a program in execution. Unlike a program which is a passive entity stored on disk, a process is an active entity loaded into RAM with a program counter, registers, and stack.

2.2 Process Control Block (PCB)

The kernel maintains a data structure for every process called the **Process Control Block (PCB)**. Key attributes stored in a PCB include:

- **Process ID (PID)**: Unique numerical identifier for the process.
- **Process State**: Current state (New, Ready, Running, Waiting, Terminated).
- **Program Counter (PC)**: Address of the next instruction to execute.
- **CPU Registers**: Accumulators, index registers, and stack pointers saved during context switches.
- **CPU Scheduling Info**: Process priority and queue pointers.

2.3 Five States of a Process

1. **New**: The process is being created.
2. **Ready**: The process is waiting to be assigned to a CPU core.
3. **Running**: Instructions are currently being executed by the CPU.
4. **Waiting (Blocked)**: The process is waiting for an I/O event or signal.
5. **Terminated**: The process has finished execution.

■ Chapter 3: Process Synchronization & Semaphores

3.1 The Critical Section Problem

When multiple concurrent processes share global data, a **Race Condition** can occur if execution order alters the outcome. A **Critical Section** is a code segment where shared variables or files are accessed.

To solve the Critical Section Problem, any synchronization solution must satisfy three requirements:

1. **Mutual Exclusion**: If process P is executing in its critical section, no other process can enter.
2. **Progress**: If no process is executing in its critical section, selection of next process cannot be postponed indefinitely.
3. **Bounded Waiting**: There must be a limit on the number of times other processes enter critical section after a request is made.

3.2 Semaphores: Counting vs Binary

A **Semaphore** is an integer variable accessed only through atomic operations: wait() (P operation) and signal() (V operation).

- **Counting Semaphore**: Value ranges over an unrestricted domain, used for resource allocation with finite instances.
- **Binary Semaphore (Mutex)**: Value ranges only between 0 and 1, used primarily for mutual exclusion locks.

■ Chapter 4: CPU Scheduling Algorithms

4.1 CPU Scheduler Overview

The **CPU Scheduler** selects a process from the ready queue and allocates the CPU. Primary metrics include CPU Utilization, Throughput, Turnaround Time, Waiting Time, and Response Time.

4.2 Common CPU Scheduling Algorithms

- 1. First-Come, First-Served (FCFS):** Non-preemptive algorithm. Simple FIFO queue. Suffers from the *Convoy Effect* where short processes wait behind long ones.
- 2. Shortest Job First (SJF):** Associates CPU burst length with each process. Gives optimal minimum average waiting time. Preemptive version is called *SRTF (Shortest Remaining Time First)*.
- 3. Round Robin (RR):** Designed for time-sharing systems. Each process gets a fixed unit of time called a **Time Quantum (q)** (typically 10-100 ms). Preemptive.
- 4. Priority Scheduling:** CPU allocated to highest priority process. Can suffer from *Starvation*, which is resolved using **Aging** (gradually increasing process priority over time).

■ Chapter 5: Deadlocks & Banker's Algorithm

5.1 What is a Deadlock?

A **Deadlock** is a situation where a set of processes are blocked because each process holds a resource and waits for another resource held by some other process.

5.2 Four Necessary Conditions for Deadlock

1. **Mutual Exclusion**: At least one resource must be held in a non-shareable mode.
2. **Hold and Wait**: A process must hold at least one resource and wait for additional resources held by others.
3. **No Preemption**: Resources cannot be preempted; they are released voluntarily only after process completion.
4. **Circular Wait**: A closed chain of processes exists, such that each process holds resources needed by the next.

5.3 Deadlock Avoidance & Banker's Algorithm

The **Banker's Algorithm** (developed by Edsger Dijkstra) is a deadlock avoidance algorithm. It tests for safety by simulating resource allocation for maximum declared claims, ensuring the system stays in a **Safe State** before granting any resource request.